

Lab Assignment 2 Report

Shariq Moghees

OL41895

Prof. Kidd

3-31-25

Program Structure

For this project, students were tasked with creating an NASM assembly program that calculates the bitwise Hamming distance between two strings. Both strings may not be longer than 255 characters, and the longer string should be truncated so that both strings have an equal number of characters.

Program Setup (.data & .bss Sections)

The **.data** section holds all the constant strings used in the program. I set up `scanf_fmt` and `printf_fmt` as formatted strings to read integers and print the final sum. `file_mode` was used to open the file in read mode('r'), and I also added a few debug format strings like "Read: %d" and "Count Read: %d" to verify values during testing. These were essential for confirming the file was being read and parsed correctly, especially since I ran into a lot of issues while debugging the loops for this project.

The **.bss** section for this program reserved space for all runtime variables. `count` stores how many integers to read, and `temp` temporarily holds each value before it's moved to `arr` (the array that holds all the integers). `file_ptr` holds the result of `open`, and `sum` tracks the running total. `Arr` is the main array for storing up to 1000 integers from the file (`arr resd 1000`).

main:

In main, I set up the stack frame and pulled the filename from `argv[1]` using `ebp+12`. I struggled with this for a while as my program kept crashing due to seg faults as I was using `esp+8`, but then I realized that UMBC's GL interface follows a different structure for the stack. I called `fopen` with read mode and stored the file pointer. From there, I called each subroutine in

order: ReadCount, ReadIntegers, CalculateSum, and PrintSum. The program flow was kept simple and linear for clarity and easier debugging.

ReadCount:

This function reads the first integer in the file and stores it in count. The very first line here was crucial because it was used to load the loop with how many iterations it needed to sum all the possible integers in the file. I used fscanf with scanf_fmt and passed the file pointer. This value directly controls how often the loop in ReadIntegers runs, so it had to be read and stored correctly to avoid logic errors.

ReadIntegers:

This was the most logic-intensive function of the program, as its primary function is to use a detailed loop (along with debug print statements) loaded by count from ReadCount, to iterate through every line in the file, read the integer, and store it in the correct place within the array's memory location. I spent a great deal of time debugging this, as I often accessed the wrong location in memory, which would end the loop prematurely and print the wrong sum. GDB was used to pinpoint precisely which lines the values of my variables were incorrect, to correct the storage in memory and accessing, so that the loop ran smoothly

The loop runs 'count' times, using ebx as the index. I read into temp for each iteration, checked that fscanf succeeded(for debugging purposes), and then moved the value into arr[ebx]. I originally stored count into ecx, but it got overwritten between calls, so I fixed that by comparing directly with [count] every loop instead

CalculateSum:

This subroutine consists of a smaller loop that iterates through all elements in arr and adds them into eax, which I then store in sum. ebx is reused as the index, just like in ReadIntegers, and the loop condition again checks against [count] to avoid relying on volatile registers that often got overwritten and lead to extensive logic errors.

PrintSum:

Finally, this subroutine simply moves the sum into eax, pushes it onto the stack with the formatted string, and calls printf to display the total sum after all the integers are added.

Reflection/Challenges/Debugging

Early on, I ran into a segmentation fault when trying to access the filename using esp+4 and esp+8, which normally works in 32-bit NASM. But after some trial and error, I realized UMBC's GL environment sets up the stack differently. Switching to ebp+12 to grab argv[1] solved the segfault and let the program open files correctly. Later, the loop in ReadIntegers kept breaking after one iteration, even though count was read correctly. Using gdb, I found that ecx—which I had loaded count into—was being overwritten during the loop, dropping to zero. Once I switched to comparing ebx directly with [count] in memory, the loop ran as expected. These bugs were initially frustrating, but debugging them taught me a lot about register volatility and system-specific behavior, which makes a world of difference in assembly.

Code:

```
section .data
    ;formatted string vars for reading the ints and printing sum
    scanf_fmt db "%d", 0
    printf_fmt db "Sum is: %d", 10, 0
    count_fmt db "Count Read: %d", 10, 0
    debug_fmt db "Read: %d", 10, 0
    ;puts the file into read mode
    file_mode db "r", 0
    fscanf_err_fmt db "fscanf faile at line %d", 10, 0

section .bss
    count resd 1      ;hold number of ints to read at the loop
    temp resd 1       ;temp var for input
    file_ptr resd 1    ;hold file location returned by fopen
    sum resd 1         ;hold total sum
    arr resd 1000      ;arr to hold intgers from file up to 1000 max

section .text
    global main
    extern fopen, fscanf, printf

main:
    push    ebp
    mov     ebp, esp

    mov     eax, [ebp+12]      ;argv pointer
    mov     eax, [eax+4]      ;get argv[1] string
    push    file_mode        ; push "rb"
    push    eax
    call    fopen
    add     esp, 8            ; clean up stack
    mov     [file_ptr], eax   ; save file pointer

    call    ReadCount
    call    ReadIntegers
    call    CalculateSum
    call    PrintSum
```

```
mov     esp, ebp
pop     ebp
ret
```

ReadCount: ;Reads the first line to set the loop (count) var in .bss

```
push    count           ; store the first int in count
push    scanf_fmt       ; "%d"
push    dword [file_ptr] ; file ptr
call    fscanf
add     esp, 12
```

```
mov     eax, [count]
push    eax
push    count_fmt       ; debug print count value
call    printf
add     esp, 8

ret
```

ReadIntegers: ;loops through with count var to initialize the array

```
xor     ebx, ebx           ;zero out index = 0
```

.loop:

```
cmp     ebx, [count]
jge     .done             ;if i >= count, terminate the loop

push    temp             ;store in temp
push    scanf_fmt
push    dword [file_ptr]
call    fscanf
add     esp, 12

cmp     eax, 1           ;check fscanf success
jne     .fscanf_fail     ;jump to the fscanf err msg subroutine

mov     eax, [temp]       ;get val from temp
mov     [arr+ebx*4], eax  ;store at arr[ebx]

push    eax             ;debug print value read
push    debug_fmt
```

```

    call    printf
    add     esp, 8

    inc     ebx                ;index increment
    jmp     .loop             ;next loop iteration

```

```

.fscanf_fail:
    push    ebx
    push    fscanf_err_fmt
    call    printf
    add     esp, 8            ;clean stack
    jmp     .done

```

```

.done:
    ret

```

CalculateSum: ;Iterate through the array and sum all the integers

```

    xor     eax, eax          ;sum
    xor     ebx, ebx          ;index

    mov     ecx, [count]      ;loop limit

.loop:
    cmp     ebx, ecx
    jge     .done

    mov     edx, [arr + ebx*4] ; get arr[ebx]
    add     eax, edx          ; sum += arr[ebx]

    inc     ebx
    jmp     .loop             ;next iteration

```

```

.done:
    mov     [sum], eax        ; store result in `sum`
    ret

```

PrintSum: ;print call to print the sum

```

    mov     eax, [sum]        ; move value into eax
    push     eax              ; second arg to printf
    push     printf_fmt       ; formatted string

```

```
call    printf
add     esp, 8
ret
```

Output Test Case(s):

```
[shariqmi@linux4 Sum_The_File]$ ./sum_the_file randomInt100.txt
Count Read: 100
Read: 85
Read: 13
Read: 90
Read: 32
Read: 4
Read: 84
Read: 9
Read: 92
Read: 26
Read: 10
Read: 34
Read: 46
Read: 99
Read: 36
Read: 61
Read: 36
Read: 86
Read: 92
Read: 17
Read: 76
Read: 83
Read: 58
Read: 3
Read: 43
Read: 4
Read: 26
Read: 55
Read: 28
Read: 36
Read: 46
Read: 82
Read: 3
Read: 100
Read: 22
Read: 6
Read: 62
Read: 23
Read: 40
Read: 26
```



```
Read: 67
Read: 19
Read: 24
Read: 11
Read: 96
Read: 91
Read: 31
Read: 82
Read: 70
Read: 47
Read: 63
Read: 45
Read: 3
Read: 83
Read: 12
Read: 17
Read: 14
Read: 16
Read: 18
Read: 67
Read: 14
Read: 58
Read: 7
Read: 13
Read: 32
Read: 17
Read: 50
Sum is: 4579
[shariqm1@linux4 Sum_The_File]$ emacs randomInt100.txt
[shariqm1@linux4 Sum_The_File]$
```